Linux Forensic Log Processing Tool

Project Plan v3 — Concept, Design, and Implementation

PART ONE — THE IDEA, IN PLAIN LANGUAGE

This part assumes no forensics background. Anyone should be able to read it and understand what we're building, what it looks at, and why it's useful.

1.1 The problem

When something goes wrong on a Linux server — someone breaks in, an account gets misused, a file goes missing, a machine starts behaving strangely — the answers are almost always already recorded somewhere on that machine. Linux writes down an enormous amount about itself: who logged in and from where, who failed to log in, who ran commands as administrator, what software was installed, what USB devices were plugged in, what scheduled jobs exist.

The problem is that this information is scattered across **dozens of files in at least four different formats**, in different places depending on which Linux distribution you're running, and much of it is unreadable by a human. Some of it is plain text. Some of it is binary data that looks like garbage in a text editor. Some of it is inside a database. Some of it is only accessible through a special command.

So in practice, when an incident happens, an investigator sits there running `grep` by hand across twenty log files, copying interesting lines into a spreadsheet, and manually trying to line up timestamps from different sources. It's slow, it's inconsistent between investigators, it's easy to miss something, and there's no record of *how* the conclusions were reached.

1.2 What we're building

A tool that does that job automatically, in two clean steps:

Step 1 — Collect. A small, dependency-free script runs on the machine under investigation. It gathers every relevant log and configuration file, takes a cryptographic fingerprint (a SHA-256 hash) of each one, and packages everything into a single sealed bundle. It reads; it never writes. It then gets off the machine.

Step 2 — Analyse. On the investigator's own computer, a second program opens that bundle, verifies every fingerprint still matches, translates all those different file formats into one common structure, stores them in a searchable database, looks for suspicious patterns, and produces a report.

The output is a single self-contained HTML file that opens in any browser with no internet connection, containing a plain-English summary of what happened, a timeline, detailed evidence tables, and an honest account of what could not be determined.

1.3 Why two separate steps

This is the most important design decision, so it's worth being clear about it.

The machine under investigation **is the evidence**. Every command you run on it changes it in some small way. If we did the analysis on the machine itself, we'd be spending hours running programs on top of the very thing we're trying to preserve — and every one of those actions becomes something we'd have to explain later.

So we touch it once, briefly, in read-only mode, and take a copy. Everything else happens on the copy. That gives us three things for free:

- **We can re-run the analysis as many times as we want.** Found a bug in our tool? Fix it and re-run. The evidence is untouched.
- **We can prove nothing changed.** The hashes taken at collection time are checked again at analysis time. If even one byte differs, we know, and we say so.
- **Someone else can verify our work.** Hand them the same bundle, they get the same result.

1.4 What we collect, and what it tells us

Everything we gather falls into eight evidence categories. Here's each one in plain terms:

1. User Accounts

Where it lives: `/etc/passwd`, `/etc/shadow`, `/etc/group`, and their backup copies.

What it tells us: Every account that exists on the machine, which ones can actually log in, which ones have administrator powers, which ones have no password at all, and when passwords were last changed. The backup copies (`/etc/passwd-`) are quietly useful — comparing them to the live files shows exactly what changed recently, for free.

Why it matters: Attackers create accounts. An account that shouldn't exist, or an ordinary account that suddenly has administrator rights, is one of the clearest signals there is.

2. Login Activity

Where it lives: `/var/log/auth.log` (Debian/Ubuntu) or `/var/log/secure` (RHEL/Fedora), the systemd journal, and three binary files — `wtmp` (successful logins), `btmp` (failed logins), `lastlog` (each user's most recent login).

What it tells us: Who logged in, when, from which IP address, over which method (SSH, console, GUI), how long the session lasted, and every failed attempt.

Why it matters: This is the front door. Hundreds of failed logins followed by one success is a password being guessed. A login at 3am from an unfamiliar address is worth a question.

3. Privilege Escalation

Where it lives: `sudo` entries inside the auth log or journal, plus `/etc/sudoers` and `/etc/sudoers.d/`.

What it tells us: Every time an ordinary user ran a command as administrator — who, when, from which directory, and the exact command — plus who is *allowed* to do so.

Why it matters: Getting in is step one; becoming root is step two. This category is where step two is recorded.

4. Persistence

Where it lives: cron jobs (`/etc/crontab`, `/etc/cron.d/`, per-user crontabs), systemd services and timers, `~/.ssh/authorized_keys`, `/etc/ssh/sshd_config`, shell startup files (`~/.bashrc`, `/etc/profile.d/`), and `/etc/rc.local`.

What it tells us: Everything configured to run automatically, and every way of getting back into the machine without a password.

Why it matters: Anyone who breaks in wants to stay in. They'll leave a scheduled job, or add their SSH key to a file. Of all eight categories this is the one investigators most often under-check, and it's the one that tells you whether the problem is actually over.

5. Software and Package Changes

Where it lives: `/var/log/dpkg.log` and `/var/log/apt/history.log` (Debian/Ubuntu), `/var/log/dnf.log` or `yum.log` (RHEL family).

What it tells us: Everything installed, upgraded, or removed, and when. The `apt/history.log` file additionally records *the exact command typed and which user ran it*, which `dpkg.log` does not.

Why it matters: Attack tools have to get onto the machine somehow. Software appearing at an odd hour, or a security tool being removed, is a story in itself.

6. Hardware and USB

Where it lives: kernel messages in `/var/log/kern.log`, `syslog`, or the journal, plus live device information under `/sys/bus/usb/`.

What it tells us: Every USB device ever plugged in — manufacturer, product, serial number, when it was connected and when it was removed.

Why it matters: Data theft frequently involves a physical device. A USB drive connected at 2am with no matching entry in the visitor log is a finding.

7. Network Configuration

Where it lives: `/etc/hosts`, `/etc/resolv.conf`, `/etc/network/interfaces` or netplan files, firewall logs (`ufw.log`, `iptables` entries), and — if we're allowed to collect live state — current connections and listening ports.

What it tells us: How the machine talks to the world, what it was listening on, and what the firewall blocked or allowed.

Why it matters: A modified DNS setting or an unexpected listening port is often the mechanism by which a compromised machine is controlled.

8. User Activity

Where it lives: shell history files (`~/.bash_history`, `~/.zsh_history`), browser history databases (Firefox, Chrome), and `~/.ssh/known_hosts`.

What it tells us: What commands people actually typed and where they browsed.

Why it matters: Sometimes this is the whole answer. Also — and this is important — the *absence* of it is a finding. A user with fifty recorded login sessions and an empty command history didn't just get shy; someone deleted it.

1.5 How collection works

The collector is a shell script. Deliberately not Python — a Python program needs Python installed, and on a stripped-down server or a rescue disk it might not be. Shell and the basic Unix commands are present on every Linux machine by definition. The script has to work on the worst machine we might meet, not the best.

It runs in five passes:

1. **Identify the machine.** Which distribution, which version, which kernel, does it use systemd. This matters because it determines where everything else lives — the same information is in `/var/log/auth.log` on Ubuntu and `/var/log/secure` on RHEL.
2. **Grab volatile things first.** If we're running on a live machine and were explicitly asked to, capture running processes and network connections *before* anything else — these change second by second, while `/etc/passwd` will still be there in ten minutes.

(This ordering is a formal forensics principle, RFC 3227's "order of volatility".)

3. **Copy the files.** Work down a configuration list of what to collect. For each file: record its existing timestamps, copy it, hash the copy, write a line into the manifest. Compressed old logs get copied still compressed, exactly as they sit on disk.
4. **Export the journal.** systemd's journal is stored in a binary format that isn't meant for outside tools to read. Rather than reverse-engineer it, we ask the machine's own `journalctl` command to export it as JSON. Doing this on the target guarantees the right version of the tool is used.
5. **Seal it.** Everything goes into one archive with a manifest listing every file, its hash, its original timestamps, what we couldn't collect, and why.

Throughout, the rule is: **if something is missing, note it and carry on.** Never abort. A machine without `journalctl` should still produce a complete bundle of everything else.

1.6 How processing works

Once the bundle is on the investigator's machine:

Verify. Re-hash everything and compare against the manifest. Anything that doesn't match is set aside and reported as an integrity failure — but the rest of the analysis continues.

Translate. This is the core idea of the whole system. Every artifact type has its own small parser, and every parser — regardless of whether it read a text log, a binary structure, or a database — outputs the same thing: a **normalized event**. One row, with a timestamp, a category, who did it, from where, a description, and a pointer back to the exact source line it came from.

Once everything is in that one shape, everything downstream becomes easy. Comparing an SSH login against a USB insertion is just a database query, because by that point they're the same kind of object. And adding support for a new log format later means writing one new parser — nothing else in the system changes.

Store. All those events go into a single SQLite database file: one file per case, no server to install, works offline, easy to hand to someone else.

Correlate. Now we look for patterns. Every rule is deliberately simple and explainable, because an investigator has to be able to defend the reasoning:

- Many failed logins from one address in a short window → probable password guessing
- A success immediately after that pattern → probable successful break-in
- An account created, then given administrator rights minutes later
- Administrator activity outside working hours
- A scheduled job created shortly after a suspicious login
- An IP address or username that has never appeared before in the entire dataset
- A log file that is suspiciously empty, or a several-hour gap in a log that's normally busy
- Timestamps that run backwards inside a single file — a sign the clock was changed or lines were inserted

Deliberately **no machine learning**. If the tool says something is suspicious, we must be able to state precisely which rule fired and on which evidence.

1.7 How results are shown

The report has two layers on the same page, because two very different people need it.

The plain-language layer, for a manager or client. Every finding answers four questions:

> **What happened** — Someone tried 47 wrong passwords for the account `admin` from 203.0.113.9 between 02:14 and 02:19, then logged in successfully at 02:19.
>
> **Why it matters** — A successful login straight after many failures usually means the password was guessed. This account can run administrator commands.
>
> **How confident we are** — High. This is recorded in two separate logs that agree with each other.
>
> **What to check next** — Confirm whether anyone legitimately uses this account at 2am, and whether 203.0.113.9 belongs to your organisation.

The report also opens with a short narrative — the flagged events written out in order as a few paragraphs of ordinary prose — so a non-technical reader understands the shape of the incident in thirty seconds, before seeing a single table.

The technical layer, for an investigator or anyone challenging our work. The same finding, with the raw log line, its byte offset, the source file and its hash, which parser produced it and at what version, which rule fired with what threshold, and the timestamp with its timezone and how confident we are in it.

And a **"What we could not determine"** section, written in the same plain language. Which artifacts were expected but absent, which timestamps were ambiguous, which categories were incomplete because the machine wasn't configured to record them. Every real forensic report has this section. Its absence is what makes an amateur report obvious.

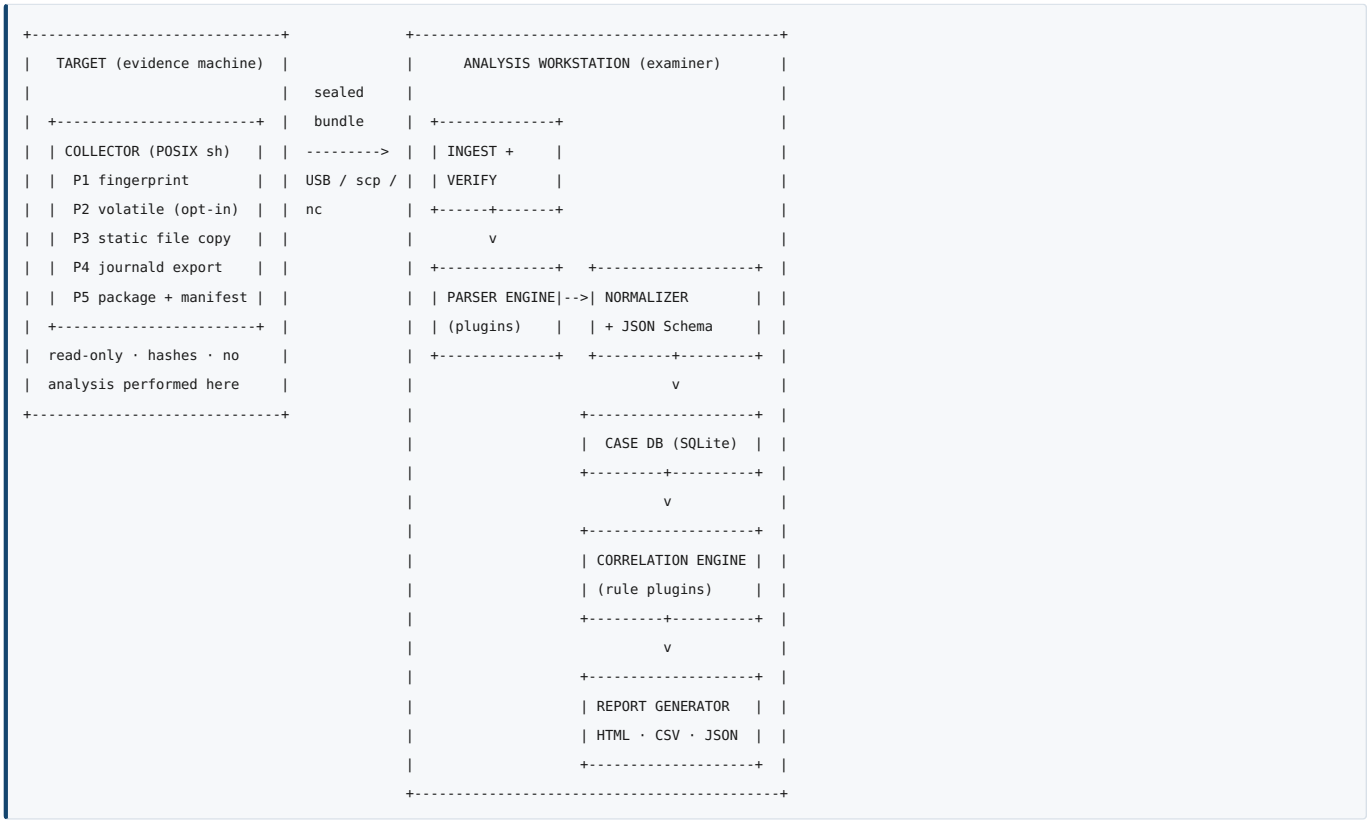
1.8 What this is not

Stated up front, because scope honesty is part of the pitch:

- Not live monitoring or intrusion detection. This is after-the-fact investigation.
- Not memory forensics or disk carving. We read logs and configuration, not RAM or deleted files.
- Not a full audit trail of every command run. Linux doesn't record that unless `auditd` was configured in advance. If it was, we collect it; if not, we say so rather than pretend.
- Not an accusation engine. It flags things for a human to judge. It never concludes.

PART TWO — TECHNICAL IMPLEMENTATION

2.1 Architecture



Five phases. Collection → Storage → Processing → Analysis → Presentation. Both team members work inside every phase; the split is by *evidence domain*, not by phase (see §2.10).

2.2 Tech stack decisions

Layer	Choice	Reasoning
Collector	**POSIX `sh` + coreutils only**	Must run where nothing is installed. No bashisms, no Python, no rsync. Test under `dash` and `busybox`, not just bash — bash will silently accept things `dash` won't.
Analyzer	**Python 3.11+**	`struct` for binary records, `sqlite3` in stdlib, `re` for text, runs on any examiner OS.
Case store	**SQLite**, one file per case	Portable, offline, no server, indexed joins across artifact types.
Text parsing	`re` with patterns loaded from **YAML grammar files**	New distro = new config file, not new code. This is the single biggest maintainability decision.
Binary parsing	`struct.unpack`, fixed 384-byte utmp records	See §2.7 gotchas.
journald	`journalctl -o json` **at collection time**	The on-disk format is undocumented and version-unstable. Use the machine's own exporter.
Report	**Jinja2 → self-contained HTML**	All CSS and JS inlined. Print stylesheet for PDF via browser.
Charts	**matplotlib**, rendered server-side to embedded SVG	No CDN, no JS chart library — the report must work with no internet.
Validation	**JSON Schema** on every event before insert	A bad parser fails its own event, not the pipeline.
Testing	`pytest` + golden fixtures + Docker attack lab	See §2.9.

Rejected: Go (weaker for ad-hoc binary/SQLite work, slower to iterate 15 parsers) · Postgres (needs a server; forensic tools must be portable and offline) · CSV as the store (no indexed cross-artifact joins) · WeasyPrint for PDF (needs Pango/Cairo native libraries — genuinely painful, and browser print does the job) · client-side CDN charts (fails offline).

2.3 Artifact → category → parser map

This table is the project's spine. It is also the traceability matrix for the final report — every row proves one requirement is covered by a named module.

#	Category	Artifacts	Format	Parser	Owner
1	**User Accounts**	`/etc/passwd`, `/etc/shadow`, `/etc/group`, `/etc/gshadow`, and `-` backup variants	colon-delimited text	`passwd_parser`	A
2	**Login Activity**	`/var/log/auth.log*`, `/var/log/secure*`	syslog text	`authlog_parser`	A
2		journald `_COMM=sshd`, `_COMM=login`, `systemd-logind` units	JSON lines	`journald_parser`	A
2		`/var/log/wtmp*`, `/var/log/btmp*`	binary, 384-byte records	`utmp_parser`	A
2		`/var/log/lastlog`	binary, sparse, UID-indexed 292-byte records	`lastlog_parser`	A
2		`/var/log/faillog`, `/var/run/faillock/*`	binary / dir	`faillog_parser` *(Extended)*	A
3	**Privilege Escalation**	`sudo:` lines in auth.log / journal	syslog text	`sudo_parser`	A
3		`/etc/sudoers`, `/etc/sudoers.d/*`	sudoers syntax	`sudoers_parser`	A
3		`/etc/pam.d/*`	text config	`pam_parser` *(Extended)*	A
4	**Persistence**	`/etc/crontab`, `/etc/cron.d/*`, `/var/spool/cron/crontabs/*`, `/etc/cron.{hourly,daily,weekly}/*`	cron syntax + file mtimes	`cron_parser`	B
4		`/var/log/cron*`, `CRON` lines in syslog	syslog text	`cronlog_parser`	B
4		`/etc/systemd/system/**`, `~/.config/systemd/user/**`, `systemctl list-timers`	unit files + text	`systemd_unit_parser`	B
4		`~/.ssh/authorized_keys*` (all users incl. root)	text, one key per line	`authkeys_parser`	B
4		`/etc/ssh/sshd_config`, `sshd_config.d/*`	key-value text	`sshdconfig_parser`	B
4		`~/.bashrc`, `~/.profile`, `/etc/profile.d/*`, `/etc/rc.local`, `/etc/ld.so.preload`	text + mtimes	`startup_parser`	B
5	**Software Changes**	`/var/log/dpkg.log*`	text	`dpkg_parser`	B
5		`/var/log/apt/history.log*`, `term.log*`	stanza text	`apt_parser`	B
5		`/var/log/dnf.log*`, `yum.log*`	text	`dnf_parser`	B
5		`/var/lib/rpm/rpmdb.sqlite` or BDB	sqlite / BDB	`rpmdb_parser` *(Stretch)*	B
6	**Hardware / USB**	`usb ... New USB device found` lines in `kern.log`, `syslog`, journal	multi-line syslog text	`usb_parser`	B
6		`/sys/bus/usb/devices/*`, `udevadm info --export-db`	key-value	`udev_parser`	B
7	**Network Config**	`/etc/hosts`, `/etc/resolv.conf`,	text/YAML	`netconfig_parser`	B

		`/etc/network/interfaces`, `/etc/netplan/*.yaml`			
7		`/var/log/ufw.log*`, `iptables` lines in kern.log	syslog text	`firewall_parser`	B
7		live `ss -tulpn`, `ip addr` (volatile)	command output	`volatile_parser`	B
8	**User Activity**	`~/.bash_history`, `~/.zsh_history` (all users)	text, usually untimestamped	`shellhist_parser`	A
8		Firefox `places.sqlite`, Chrome `History`	SQLite	`browser_parser` *(Extended)*	A
8		`~/.ssh/known_hosts`	text	`knownhosts_parser`	A
—	**Environment** (feeds everything)	`/etc/os-release`, `/etc/localtime`, `/etc/timezone`, `/etc/machine-id`, `timedatectl`	mixed	`env_parser`	A
—	**Filesystem timeline** *(Extended)*	`find -printf` MACB body file over `/etc`, `/usr/bin`, `/usr/sbin`, `/tmp`, `/home`, web roots	body file	`fstimeline_parser`	B
—	**Audit**	`/var/log/audit/audit.log*`	audit text	*collected, not parsed* — stated in limitations	B

Collected but never parsed on the target: everything. Parsing is Phase B, always.

2.4 Phase 1 — Collection

2.4.1 Constraints

- POSIX `sh` only. No bashisms. Verify with `dash` and `busybox sh`.
- Takes a `$ROOT` variable defaulting to `/`, so live collection and offline collection against a mounted image are **the same code path**. Offline mounts must be `ro,noatime`.
- Never modifies source files. Never uses `cat > file` on the target tree.
- Every failure is per-artifact: log the reason, continue.

2.4.2 Step-by-step workflow

P1 — Fingerprint. Read `$ROOT/etc/os-release` → `ID`, `VERSION_ID`. Fallback chain: `/etc/redhat-release` → `/etc/debian_version` → `uname -a`. Detect init system via `/run/systemd/system`. Resolve `/etc/localtime`'s symlink target for the timezone. Check privilege level and **immediately warn** which artifacts will be unreadable if not root (`/etc/shadow`, `/var/log/btmp`, other users' home directories). Write `manifest.json` header: distro, version, kernel, hostname, machine-id, timezone, init system, collector version + git commit, case number, operator name, collection start time in UTC.

P1b — Contamination record. Write our own footprint into the manifest: operator username, source IP, TTY, PID, collector start/end times. The analyzer uses this to auto-tag our own session's `wtmp` and `auth.log` entries as `tool_generated_flag=true` so they never show up as findings. *Without this the tool reports its own operator as a suspicious 3am login.*

P2 — Volatile snapshot *(live mode + explicit `--include-volatile` only, runs FIRST).* `ps aux`, `ss -tulpn` (fallback `netstat -tulpn`), `ip addr`, `mount`, `lsblk -f`, `blkid`, `uptime`, `who`, `w`, `last`, `systemctl list-timers`, `docker ps -a` if present. Each command's stdout captured to its own file under `collected/volatile/`, each hashed, each with its own capture timestamp. Kept visually separate in the report because it reflects the moment of collection, not history.

P3 — Static file collection. Iterate `artifacts.yaml`. Per entry: path or glob, category tag, required/optional, whether to glob rotated variants (`auth.log*`). For each matched file:

1. `stat` it and record atime/mtime/ctime/size/mode/owner **before touching it**. This survives even though reading the file will clobber the live atime under `relatime`.
2. Copy with `cp -p`, preserving the directory structure under `collected/`. Use `cp --sparse=always` for `lastlog`. If `cp -p` is unavailable, degrade to `cat < src > dst` and log the degradation explicitly.
3. `sha256sum` **the copy** — this is the authoritative evidence hash.
4. `stat` the source again. If size or mtime changed, set `source_was_active: true` on that artifact. This is an observation, not an error — active logs grow during collection and that's normal.
5. Append to `hash_manifest.csv`: `original_path, sha256, size, mode, owner, atime, mtime, ctime, source_was_active, status`.

Compressed rotated logs are copied **still compressed**. Hashing the compressed bytes attests to the file exactly as it existed; decompressing first would attest to something we created.

P4 — journald export. `journalctl --list-boots`, then per boot `journalctl -b <id> -o json --no-pager > collected/journal/boot-<n>.json`. Record `journald_persistent: true/false` by testing `-d $ROOT/var/log/journal` vs only `/run/log/journal` existing — if only the latter, the journal is wiped on reboot and that becomes an automatic caveat in the report. Hash each export. If `journalctl` is missing, log and continue.

P5 — Package and seal. `tar --numeric-owner -cf bundle.tar collected/`, hash the tar, *then* optionally gzip and hash the gzip — both layers hashed so integrity is checkable at either. Finalize `manifest.json` with the full hash list, end timestamp, and a **complete list of expected-but-missing artifacts with reasons**. Transport is not the collector's problem: it produces a bundle and exits. USB, `scp`, or `nc` all work; the hashes cover the transfer either way.

2.4.3 `artifacts.yaml` shape

```
- id: auth_log
  category: login_activity
  paths: ["/var/log/auth.log*", "/var/log/secure*"]
  distros: [debian, ubuntu, rhel, centos, fedora]
  required: false          # false because journald-only systems legitimately lack it
  glob_rotated: true
  notes: "Absent on Debian 12 and minimal cloud images — journald is primary there"
```

2.5 Phase 2 — Storage: the normalized event

Freeze this in week 1, before writing anything else. It is the contract that lets two people work without blocking each other.

```
event_id             UUID
case_id              str
host_id              str            # multi-host cases in one DB
event_hash           sha256         # dedupe: hash(host_id, source_path, offset, raw_line)
event_kind           enum           # "event" (something happened at a time)
                                    # "state_finding" (something is true of the config)
timestamp_utc        datetime|null
timestamp_local      datetime|null
timestamp_tz         str            # "Asia/Karachi"
tz_source            enum           # etc_localtime | log_offset | assumed_utc
timestamp_confidence enum           # exact | year_inferred | unknown
category             str            # the eight from §2.3
subcategory          str
actor_user           str|null
actor_uid            int|null
actor_process        str|null
source_ip            str|null
source_host          str|null
description          str
severity             enum           # info | low | medium | high
source_artifact_path str
source_artifact_sha256 str
raw_line             str            # truncated at 2 KB
raw_line_offset      int
parser_name          str
parser_version       str
tool_generated_flag  bool
notes                str|null
```

Indexes: `timestamp_utc`, `category`, `actor_user`, `source_ip`, `host_id`. These are exactly the join keys the correlation layer needs — nothing else is indexed.

Ingest module workflow: extract bundle → `cases/<case_id>/raw/` → re-hash every file against `hash_manifest.csv` → mismatches moved to `raw/_integrity_failed/` and logged, analysis continues → write `case_metadata.json` merging collector manifest with examiner name, ingest time, analyzer version. `raw/` is enforced read-only **in code** (always opened `'rb'`, never written), with a final re-verification pass at the end proving nothing changed. `chmod 444` is best-effort only — it's a no-op on Windows examiner machines.

Determinism guarantee: the *canonical JSON/CSV export* of the case is byte-identical across runs, verified by hashing the export. Not the SQLite file itself — page layout and freelists aren't contractually stable, so don't promise something SQLite doesn't give you.

2.6 Phase 3 — Processing: the parser engine

2.6.1 Plugin interface

```
class BaseParser:
    name: str
    version: str
    artifact_category: str
    applies_to: list[str]           # glob patterns claimed, e.g. ["var/log/auth.log*"]

    def can_parse(self, path, distro_profile) -> bool: ...
    def parse(self, path, context) -> Iterator[NormalizedEvent]: ...
```

Auto-discovered from a registry directory. Each plugin runs inside its own try/except boundary; failures write a full traceback to `parser_errors.log` and the engine continues. The run produces per-artifact success/fail/skip counts that go straight into the report's methodology section — analysts need to know what *wasn't* processed, not only what was.

A single log line may yield **more than one event** (a `sudo` line is both privilege escalation and timeline activity). Parsers yield freely; they don't force one line into one bucket.

Encoding: all text artifacts opened `encoding='utf-8', errors='surrogateescape'`, with a `had_decode_errors` flag per artifact. Log files routinely contain non-UTF-8 bytes and a naive `open()` will raise mid-file and silently lose the remainder.

2.6.2 The timestamp engine (hardest single component — budget extra time)

Classic syslog lines (`Mar 14 02:19:07`) have **no year and no timezone**. Resolution order:

1. **Timezone** from the collected `/etc/localtime` symlink target. If missing, fall back to `/etc/timezone`, then to `assumed_utc` — and set `tz_source` accordingly so the report can flag it. *A tool that normalizes to UTC without collecting the timezone is silently wrong by hours; this is the most dangerous quiet bug in the project.*
2. **Year** inferred from: file mtime as an upper bound; rotation ordering (`auth.log.3` predates `auth.log.1`); and cross-checking any ISO-8601 lines in the same file (rsyslog can be configured either way).
3. Anything inferred is marked `year_inferred`, never presented as certain.
4. Watch the **December→January rollover** inside one rotated file. Dedicated unit test.

2.6.3 Grammar files

```
# grammars/debian.yaml
sshd_failed_password:
  pattern: '^(?P<ts>\w{3}\s+\d+\s[\d:]{8})\s\S+\sshd\[(?P<pid>\d+)\]:\s
            Failed password for (?:invalid user )?(?P<user>\S+)
            \sfrom\s(?P<ip>\S+)\sport\s(?P<port>\d+)'
  category: login_activity
  subcategory: failed_login
  severity: low
```

New distro or new log format = new YAML file. No Python changes.

2.7 Things to watch out for (learned-the-hard-way list)

#	Trap	Handling
1	**Hashing the source before copying fails on live systems.** `auth.log` grows mid-collection, so source hash ≠ copy hash, and a naive design calls that an integrity failure on exactly the files you care most about.	Copy first, hash the copy. Compare `stat` before/after and record `source_was_active` as an observation.
2	**"Read-only" is not literally achievable.** `relatime` updates atime on read; your SSH login writes `wtmp` and `auth.log`; your shell may write `.bash_history`.	`stat` before reading (preserves the original atime in the manifest) + the contamination record from P1b + `ro,noatime` for offline mode.
3	**GNU extensions break the POSIX promise.** `cp --preserve=`, `rsync`, `stat -c` are not universal; busybox has neither. `--preserve=ownership` silently fails without root.	`cp -p` only, capture ownership into the manifest rather than relying on the filesystem, documented degradation path.
4	**No timezone collected = every timestamp wrong.**	§2.6.2.
5	**The 32/64-bit utmp offset table is largely a non-problem.** glibc pins `ut_tv` to two `int32` for compatibility, so `struct utmp` is **384 bytes on both x86 and x86_64**. Building per-arch offset tables solves a problem you mostly don't have.	Ship one 384-byte layout. Validate with `filesize % 384 == 0` and a sanity check that decoded timestamps land in a plausible range; otherwise log unsupported-layout. Spend the saved effort on the *real* edge case: **musl/Alpine, where `wtmp`/`btmp` are stubs and frequently empty or absent**, and containers where they're meaningless.

6	`lastlog` **is sparse and UID-indexed.** Record offset = `uid * 292`. With a UID like 65534 present the logical size is ~19 MB while on-disk it's a few KB. A naive copy inflates it; a naive sequential read yields tens of thousands of empty records.	`cp --sparse=always`; parse by seeking to `uid*292` for UIDs found in `/etc/passwd` only; skip zero-time records. Note `lastlog` is being retired in favour of `lastlog2` on newer distros — record absence as a finding, not a crash.
7	`/var/log/auth.log` **may not exist at all.** Debian 12 and most minimal/cloud images don't install rsyslog; everything is in journald. Building the pipeline around auth.log and discovering this in week 5 is a bad afternoon.	journald parser is **Core, equal priority to auth.log**. Test on Debian 12 in week 3.
8	**Browsers hold an exclusive lock on their SQLite files.** Opening the live DB corrupts state or fails. Even the ingested copy needs write access for SQLite's rollback journal.	Copy to a temp location and open the copy. Chrome timestamps are **WebKit epoch — microseconds since 1601-01-01**, not Unix. Separate, explicitly-tested conversion function.
9	**USB kernel messages span multiple lines.** `New USB device found, idVendor=..., idProduct=...` then Manufacturer/Product/SerialNumber on *following* lines, keyed by the `usb N-M` bus-port ID.	Small stateful buffer keyed by bus-port ID, stitching into one event; match the later `USB disconnect` line to compute connection duration.
10	**Shadow's "last changed" field is days-since-epoch, not seconds.** Classic off-by-one source.	Dedicated unit test with known values.
11	**Re-collecting the same host double-counts events.**	`event_hash` dedupe key on insert.
12	**Non-UTF-8 bytes in logs crash the parser mid-file.**	`errors='surrogateescape'`, `had_decode_errors` flag.
13	**Privacy.** `/etc/shadow` holds crackable hashes; browser history is deeply personal.	`--redact-secrets` mode storing a placeholder for password fields, plus an **Authorisation & Privacy** section in the report stating scope of consent.
14	**Large logs.** Multi-GB syslog files.	Stream line-by-line, never `readlines()`. Batch inserts inside a transaction (~5000 rows).
15	`nc` **transport is unauthenticated and unencrypted.**	Document it; hashes detect tampering but don't prevent interception. Prefer `scp`.

2.8 Phase 4 & 5 — Analysis and presentation

2.8.1 Correlation rules

Same plugin pattern as parsers, but operating on the DB instead of files. One shared helper does the generic work — "*return all events within ±N minutes of this event*" — and every rule is built on top of it. Less code, more rules.

Attack-behaviour rules - Brute force: > N failed logins (default 5) from one IP in a rolling window (default 5 min), with the full list of attempted usernames - Success immediately following that pattern from the same IP → high severity - Off-hours privileged activity (configurable business hours) - New account, then privilege grant within a short window - Scheduled job or systemd timer created near a suspicious login - `authorized_keys` entry whose file mtime falls inside the incident window - Unknown USB vendor — checked against a **bundled offline** vendor ID database, never a live API call. Unknown = flagged for review, never auto-labelled malicious.

First-seen analysis — for every source IP and username, when did it first appear anywhere in the dataset? Two lines of SQL, and often more useful than any threshold rule.

Evidence-integrity rules (the ones that make a report look professional) - `wtmp` / `btmp` / `lastlog` present but zero-length or implausibly small - `.bash_history` missing, empty, or symlinked to `/dev/null` for a user with recorded sessions - Rotation sequence with a missing number - Timeline gaps: N hours with zero events in a log that otherwise averages X/hour - Timestamps running backwards inside one file → clock manipulation or inserted lines - journald non-persistent → automatic report-level caveat wherever journald was the sole source

State findings (`event_kind = state_finding`) - Any UID 0 account other than `root` - Passwordless accounts; accounts with a shell but no password - Users in `sudo` / `wheel` / `adm` / `docker` (docker group membership ≈ root) - `PermitRootLogin yes`, `PasswordAuthentication yes` in `sshd_config` - `/etc/ld.so.preload` non-empty (userland rootkit indicator) - Diff between `/etc/passwd` and `/etc/passwd-`

Every rule returns **both** `technical_detail` and a `plain_summary` with the four keys from §1.7. A rule that can't produce a plain-English line doesn't ship — this is enforced by the rule interface, not by review.

2.8.2 Report structure

1. Authorisation, scope, and privacy note
2. Executive summary — auto-generated from flagged findings, plain language
3. Narrative timeline — flagged events as ordered prose
4. Findings, each with both layers side by side

5. Eight per-category detail sections, sortable/filterable tables (JS **inlined**, no CDN)
6. Timeline chart — matplotlib SVG, event density across the case span, anomalies highlighted
7. Methodology — tool + plugin versions, full hash table (collection hash vs verified-at-ingest hash), parser success/fail/skip counts
8. **What we could not determine** — expected-but-absent artifacts, ambiguous timestamps, incomplete categories, all in plain language
9. Glossary — one line each for sudo, wtmp, journald, cron, SSH key, etc.

Exports: self-contained HTML (primary), plus per-category CSV/JSON for import into other timeline tools. Severity never conveyed by colour alone — accessibility, and it must print.

2.9 Testing

- **Unit tests per parser** against golden fixture files: real sanitized logs from Ubuntu 22.04/24.04, Debian 12, and one RHEL-family box. Log parsing bugs are almost always format-drift bugs; golden files catch them.
- **Corruption injection suite**: truncate, zero out, and inject garbage into fixture files to prove "fail loud per-artifact, never fail whole" actually holds.
- **The attack lab.** A Dockerfile target plus a `scenario.sh` that *performs* a known sequence: N failed SSH logins → one success → `useradd` → add to sudo → drop an `authorized_keys` entry → install a cron job → wipe `.bash_history`. This produces **ground truth**.
- **Scoring script** comparing detections against ground truth: "*detected 9 of 10 injected events; 1 false negative (X, because Y); 2 false positives.*"

That last item is the highest marks-per-hour work in the entire project. Almost nobody at this level evaluates their own tool quantitatively, and one slide reporting a measured detection rate with a named, explained miss outranks three extra parsers.

2.10 Work division — both people in every phase

Split by **evidence domain**, not by phase, so neither person waits on the other and both touch collection, storage, processing, analysis, and presentation.

Person A — Identity & Access domain *Who exists on this machine, who got in, and who became root.* Categories 1, 2, 3, 8, plus environment.

Person B — System & Activity domain *What runs, what was installed, what was plugged in, how it talks to the world.* Categories 4, 5, 6, 7, plus filesystem timeline.

Shared infrastructure is split so each owns roughly half of every phase:

Phase	Person A owns	Person B owns
1 · Collection	Collector core engine: pre-flight fingerprint, privilege check, contamination record, `stat`-then-copy-then-hash loop, degradation logging. Their half of `artifacts.yaml`.	journald extractor, volatile snapshot module, udev/USB metadata capture, packaging + manifest finalization, transport helpers. Their half of `artifacts.yaml`.
2 · Storage	`NormalizedEvent` JSON Schema, SQLite DDL + indexes, insert/validate layer. **Week 1 deliverable — B depends on it.**	Ingest + integrity verification, quarantine, `event_hash` dedupe, canonical deterministic export + its hash.
3 · Processing	Parser plugin framework + registry + error isolation. Identity parsers: passwd/shadow/group, auth.log, journald, sudo/sudoers, utmp/wtmp/btmp, lastlog, shell history, known_hosts.	Grammar engine + timestamp/timezone/year-inference engine. Activity parsers: cron, cron log, systemd units, authorized_keys, sshd_config, startup files, dpkg/apt/dnf, USB kernel + udev, network config, firewall.
4 · Analysis	Correlation engine core + the generic ±N-minute join helper. Rules: brute force, brute-force→success, off-hours privileged, new-account→privilege, first-seen IP/user, UID-0 and passwordless state findings.	Rules: persistence correlation, unknown-USB vendor, package-change correlation, evidence-integrity rules (zero-length logs, wiped history, timeline gaps, backwards timestamps), sshd_config and ld.so.preload state findings.
5 · Presentation	Report data layer (the queries feeding each section), technical-layer rendering, methodology + hash table + limitations sections.	Jinja2 templates and CSS, plain-language layer rendering, narrative timeline generator, matplotlib chart, glossary, CLI + packaging.
6 · Testing	Fixture corpus + unit tests for their parsers, corruption-injection suite.	Docker attack lab, `scenario.sh`, ground-truth scoring script, unit tests for their parsers.

Done jointly, not split: the §2.5 schema (agree it, freeze it, changes need both to sign off) · repo hygiene (one repo, protected `main`, feature branches, every PR reviewed by the other person) · the §2.3 traceability matrix, each filling their own rows · the final report and demo.

The unblocking trick. On day 3, one of you writes `tools/make_synthetic_case.py`: emits a few thousand plausible `NormalizedEvent` rows including a scripted attack. Both of you can then build correlation rules and report sections against a full database **before a single real parser exists**. Without it, everything downstream of parsing waits three weeks. This one script is the difference between the project finishing and not.

2.11 Scope tiers

CORE — must ship. Nothing else starts until this runs end to end. Collector (P1, P1b, P3, P4, P5) · ingest + verification · schema + DB · parser framework · passwd/shadow, auth.log, journald, sudo/sudoers, wtmp/btmp, cron, dpkg + apt, authorized_keys + sshd_config, USB kernel lines · correlation rules listed in §2.8.1 under attack-behaviour, first-seen, and evidence-integrity · HTML report with both layers · `collect` and `analyse` CLI · fixtures + attack lab.

EXTENDED — after Core is green, in this order. Filesystem MACB timeline · volatile snapshot · browser history · lastlog/faillog · systemd timers · offline disk-image mode · RHEL grammar file · `.bash_history` correlation · PAM parser.

STRETCH — only if genuinely ahead. PyInstaller frozen binary · RPM DB parser · container log parsing · Timesketch/L2T CSV export · GPG-signed manifests · multi-host case comparison.

Out of scope, stated in the report: auditd-dependent analysis (execve tracking, file modification history) unless auditd was pre-configured · memory forensics · disk carving · network packet capture · live monitoring.

2.12 Milestones and gates

Adjust week numbers to your deadline; the **order and the gates** are what matter.

Week	Person A	Person B	Gate
1	Repo, schema + DB layer, collector pre-flight	`artifacts.yaml`, synthetic case generator, report skeleton	**Schema frozen.**
2	Collector copy loop + contamination record; parser framework	journald extractor, packaging; ingest + verification	**G1: a synthetic case renders a real HTML report.**
3	auth.log + journald + passwd parsers end-to-end	cron + dpkg/apt parsers; timestamp/timezone engine	**G2: real collected data → real report.** If this slips, cut Extended immediately.
4	wtmp/btmp, sudo, sudoers; identity correlation rules	authorized_keys, sshd_config, USB, systemd; activity + integrity rules	Docker attack lab running against the real collector.
5	Technical report sections, methodology, fixtures, corruption tests	Plain-language layer, narrative, chart, CLI, scoring script	**G3: feature freeze. Detection rate measured.**
6	Extended items by priority; docs; traceability rows	Extended items; final report; demo rehearsal	Ship.

Cut rule: if G2 hasn't happened by end of week 3, drop all Extended work and every Core parser beyond passwd, auth.log, journald, wtmp, and cron. A narrow tool that works end to end beats a wide one that doesn't run.

2.13 Definition of done

- [ ] `collect` runs cleanly on Ubuntu 22.04, Ubuntu 24.04, **Debian 12** (the no-rsyslog case), and one RHEL-family box
- [ ] `collect` degrades gracefully with no `journalctl`, no `python3`, and non-root — clear logged reasons, never an abort
- [ ] `analyse` on a bundle containing one deliberately corrupted file quarantines it and completes
- [ ] Attack scenario detection rate measured and stated, with named and explained false negatives
- [ ] Report opens correctly with the network **actually disabled** — verified, not assumed
- [ ] Same bundle analysed twice → identical canonical export hash
- [ ] Every finding carries both the technical detail and the four plain-language fields
- [ ] Traceability matrix maps every requirement to a module and a report section
- [ ] Limitations section is specific and honest: auditd, containers, musl targets, timestamp ambiguity, privacy scope

2.14 Risk register

Risk	Likelihood	Mitigation
Test distro has no `auth.log` at all	**High**	journald parser is Core, not Extended. Test Debian 12 in week 3.
Person A blocked on parsers, Person B blocked on schema	**High if ignored**	Week-1 schema freeze + synthetic case generator. This is the entire reason §2.5 and the unblocking trick exist.
Scope creep back toward "every parser"	High	§2.11 is a contract. Nothing moves up a tier without something moving down.
Syslog year inference wrong across Dec→Jan	Medium	Dedicated fixture crossing New Year; surface as `year_inferred`, never silently guess.

Integration hell in the final week	Medium	Three named gates, each producing something demoable. Never integrate for the first time in week 6.
utmp layout mismatch on an unusual target	Low–Medium	Size-modulo + sane-date validation, clear per-artifact failure. Alpine/musl documented as unsupported.